Practical No. 1

AIM: Implement Linear Regression (Diabetes Dataset)

Code:

```
import numpy as np
import pandas as pd
from sklearn import metrics

data_set= pd.read_csv('diabetes.csv')
data_set
X= data_set.iloc[:, :-1].values

Y= data_set.iloc[:, -1].values

from sklearn.model_selection import train_test_split
X_train, X_test, Y_train,Y_test = train_test_split(X,Y,test_size =
0.3,random_state= 99)

from sklearn import linear_model
model = linear_model.LinearRegression()
model.fit(X_train, Y_train)
y_pred = model.predict(X_test)
result = pd.DataFrame({'Actual': Y_test, 'Predict' : y_pred})

result


mse= metrics.mean_squared_error(Y_test, y_pred)
print("Mean Squared Error:", mse)

rmse= np.sqrt(mse)
print("Root Mean Squared Error:", rmse)



accuracy = model.score(X_test,Y_test)*100

print("Accuracy: ",accuracy)
```

Practical No. 2

AIM: Implement Logistic Regression (Iris Dataset)

Code:

```
import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import mean_squared_error

iris= datasets.load_iris()
iris["feature_names"]
df= pd.DataFrame(iris["data"],columns= iris["feature_names"])
df.head()
df.describe()
x= df
y= iris["target"]
print(x)
print(y)
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size = 0.3, random_state= 0)
model = LogisticRegression()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)

mse= mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
rmse= np.sqrt(mse)
print("Root Mean Squared Error:", rmse)
```

```
accuracy = model.score(x_test,y_test)*100
print("Accuracy: ",accuracy)
```

Practical No. 3

AIM: Implements Multinomial Logistic Regression (Iris Dataset)

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

iris = datasets.load_iris()
x= iris.data
y= iris.target

from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size = 0.2, random_state= 42)

multi_logisticreg = LogisticRegression(solver='lbfgs')
multi_logisticreg.fit(x_train, y_train)

y_pred = multi_logisticreg.predict(x_test)
print('Multinomial logistic regression accuracy:',accuracy_score(y_test, y_pred))
```

```
Multinomial logistic regression accuracy: 1.0
```

```
conf_matrix = confusion_matrix(y_test, y_pred)
fig, axs = plt.subplots()
axs.imshow(conf_matrix, cmap=plt.cm.Blues)
axs.set_title("Multinomial logistic regression")
axs.set_xlabel('Predicted labels')
axs.set_ylabel('True labels')
axs.set_xticks(np.arange(len(iris.target_names)))
axs.set_yticks(np.arange(len(iris.target_names)))
axs.set_xticklabels(iris.target_names)
axs.set_yticklabels(iris.target_names)
plt.show()
```

Practical No. 4

AIM: Implement SVM classifier (Iris Dataset)

Code:

```
import numpy as np
import pandas as pd
from sklearn import datasets

iris = datasets.load_iris()
iris["feature_names"]
```

```
df= pd.DataFrame(iris["data"],columns= iris["feature_names"])
df.head()
df.describe()

x= df
y= iris["target"]
print(x)
print(y)
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size = 0.3, random_state= 0)

from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
x_train = sc.fit_transform(x_train)
x_test = sc.transform(x_test)

from sklearn.svm import SVC
svc_clf = SVC(kernel = 'linear',random_state=0)
svc_clf.fit(x_train,y_train)

y_pred = svc_clf.predict(x_test)
print(y_pred)
from sklearn.svm import SVC
svcclassifier = SVC(kernel = 'poly', random_state=0)
svcclassifier.fit(x_train,y_train)
y_pred = svc_clf.predict(x_test)
print(y_pred)
```

```
from sklearn.svm import SVC
svcclassifier = SVC(kernel = 'rbf', random_state=0)
svcclassifier.fit(x_train,y_train)
y_pred = svc_clf.predict(x_test)
print(y_pred)
from sklearn.metrics import mean_squared_error
accuracy = svc_clf.score(x_test,y_test)*100
print("Accuracy:" ,accuracy)
```

Practical No. 5

AIM: Train and fine-tune a Decision Tree for the Moons Dataset

Code:

```
import numpy as np
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from matplotlib import pyplot as plt
from sklearn import tree

dataset=make_moons(n_samples=10000, shuffle=True, noise=0.4,
random_state=42)
x,y=dataset
print(x)
print(y)
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2,random_state=42)
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
x_train = sc.fit_transform(x_train)
```

```
x_test = sc.transform(x_test)
print(x_train)
print(y_train)

from sklearn.tree import DecisionTreeClassifier
classifier = DecisionTreeClassifier(criterion = 'entropy', random_state = 0)
classifier.fit(x_train, y_train)
print(classifier.predict(sc.transform([[2,500]])))

y_pred = classifier.predict(x_test)
from sklearn.metrics import accuracy_score
accuracy_score(y_test, y_pred)

from sklearn.metrics import  confusion_matrix
cm=np.array(confusion_matrix(y_test,y_pred))
cm
                    array([[816, 197],
                           [177, 810]], dtype=int64)

from sklearn.metrics import plot_confusion_matrix
plot_confusion_matrix(classifier, x_test, y_test)
plt.show()
tree.plot_tree(classifier)
```

Practical No.6

AIM: Train an SVM regressor on the California Housing Dataset

Code:

```
import pandas as pd
from sklearn.datasets import fetch_california_housing

housing = fetch_california_housing()
housing ["feature_names"]

df = pd.DataFrame(housing["data"], columns = housing["feature_names"])
df.head()
x =df
y = housing["target"]

from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.3,random_state=0)

from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
x_train = sc.fit_transform(x_train)
x_test = sc.transform(x_test)

from sklearn.svm import SVR
model_svr = SVR(kernel="rbf")
model_svr.fit(x_train,y_train)
y_pred = model_svr.predict(x_test)
print(y_pred)
from sklearn.metrics import mean_squared_error
accuracy = model_svr.score(x_test,y_test)*100
print("Accuracy: " ,accuracy)
```

Practical No. 7

AIM: Implement MLP for classification of handwritten digits (MNIST Dataset)

Code:

```
pip install tensorflow
pip install keras

from keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from keras.models import Sequential
from keras.layers import Dense,Dropout
import matplotlib.pyplot as plt

(X_train,Y_train),(X_test,Y_test)=mnist.load_data()
plt.imshow(X_train[0])
plt.show()
plt.imshow(X_train[1])
plt.show()

print(X_train[0].shape)
X_train = X_train.reshape(60000,28,28,1)
X_test = X_test.reshape(10000,28,28,1)
                                (28, 28)
Y_train=to_categorical(Y_train)
Y_test=to_categorical(Y_test)
print(Y_train[0])
```

```
import numpy as np
image_size = X_train.shape[1]
input_size = image_size * image_size
input_size


X_train = np.reshape(X_train, [-1, input_size])
X_train = X_train.astype('float32') / 255
X_test = np.reshape(X_test, [-1, input_size])
X_test = X_test.astype('float32') / 255


model=Sequential()
model.add(Dense(256,activation='relu',input_dim=input_size))
model.add(Dense(256,activation='relu'))
model.add(Dense(10,activation='softmax'))
model.summary()
model.compile(optimizer='adam',loss='categorical_crossentropy',metrics=['accurac
y'])
model.fit (X_train,Y_train,epochs=20,batch_size=128)
loss, acc = model.evaluate(X_test, Y_test, batch_size=128)
print("\nTest accuracy: %.1f%%" % (100.0 * acc))
```

Practical No. 8

AIM: Classification of images of clothing using Tensorflow (Fashion MNIST dataset)

Code:

```
import tensorflow as tf
```

```
import numpy as np
import matplotlib.pyplot as plt

fashion_mnist = tf.keras.datasets.fashion_mnist
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()

class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

train_images.shape
test_images.shape
plt.figure()
plt.imshow(train_images[0])
plt.colorbar()
plt.grid(False)
plt.show()
train_images = train_images / 255.0
test_images = test_images / 255.0

plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i], cmap=plt.cm.binary)
    plt.xlabel(class_names[train_labels[i]])
```

```
plt.show()
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10)
])
model.compile(optimizer='adam',loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),metrics=['accuracy'])

model.fit(train_images, train_labels, epochs=10)
test_loss, test_acc = model.evaluate(test_images,  test_labels, verbose=2)
print('\nTest accuracy:', test_acc)
```